

**VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING**



**REPORT
CAPSTONE PROJECT
SEMESTER 251 ACADEMIC YEAR 2025-2026**

SLURM AGENT SPECILIST

MAJOR: COMPUTER SCIENCE

SUPERVISOR(s): Assoc.Prof. Thoại Nam

——o0o——

STUDENT: Nguyễn Phúc Thanh Danh - 2252102

HO CHI MINH CITY, December 2024

Instructor's Signature

_____ Date: _____

Assoc.Prof. Thoại Nam (Instructor)

Faculty of Computer Science and Engineering

Declaration of Authenticity

We declare that we solely conducted this specialized project, under the supervision of Assoc.Prof. Thoại Nam at the Faculty of Computer Science and Engineering, Vietnam National University - Ho Chi Minh City University of Technology.

We have taken care to properly acknowledge and document all external sources and references used in the project.

If there is any instance of plagiarism, we are ready to accept the consequences. Ho Chi Minh City University of Technology - Vietnam National University HCMC will not be held responsible for any copyright violations that may have occurred during my research.

Ho Chi Minh City, December 2024

Authors,

Acknowledgement

We would like to express my appreciation to Assoc.Prof. Thoại Nam for his invaluable guidance. Our research has greatly benefited from his deep knowledge, perceptive criticism, and constant support.

In addition, we would like to extend our gratitude to my family. Their unwavering faith in our abilities and constant encouragement have been my pillars of strength. Their belief in my potential has been a constant source of motivation and resilience. We are forever grateful for their love and support.

Abstract

High-Performance Computing (HPC) clusters have become essential infrastructure for scientific research, engineering simulations, and large-scale data processing. The Slurm workload manager, deployed on the majority of the world’s top supercomputers, provides sophisticated resource management capabilities but presents significant usability challenges due to its complex command-line interface. This thesis addresses these challenges by designing and implementing an intelligent agent system that provides a natural language interface for Slurm cluster management.

The proposed Slurm Agent leverages Large Language Models (LLMs) combined with the Model Context Protocol (MCP) to bridge the gap between user intent and system operations. The system implements a multi-agent architecture where specialized sub-agents handle different aspects of cluster interaction: an Analysis Sub-Agent for read-only queries, an Action Sub-Agent for job management, plus auxiliary tools for data visualization and documentation retrieval. Through MCP-based tool integration, the agent executes Slurm commands, interprets results, and presents information in accessible formats.

Key contributions of this research include: (1) a natural language interface enabling users to query cluster state without command syntax knowledge, (2) an MCP server implementation extending the protocol ecosystem to the HPC domain, (3) a multi-agent architecture with handoff mechanisms for specialized task handling, (4) integrated visualization capabilities for cluster metrics, and (5) a scenario-grid evaluation framework that scores architecture-level behavior (tool usage, routing, HITL, and state transitions).

Experimental evaluation on the current 367-case benchmark reports a pass rate of 64.6% at a strict threshold (≥ 0.80), an average overall score of 0.775, Behavioral Agreement Rate (BAR) of 0.684, Cross-Scenario Robustness (CSR) of 0.825, and average re-

sponse latency of 12.92 seconds. The results show strong performance in read-oriented workflows and identify reliability gaps in complex multi-step and safety-sensitive operations.

The implementation provides a foundation for continued research in AI-assisted HPC operations, demonstrating the viability of LLM-based interfaces for complex system management while identifying concrete priorities for future enhancement, including safety hardening, submission workflow reliability, and production-cluster validation.

Keywords: High-Performance Computing, Slurm, Large Language Models, Model Context Protocol, Multi-Agent Systems, Natural Language Interface

List of Abbreviations

Abbreviation	Description
API	Application Programming Interface
CLI	Command-Line Interface
CoT	Chain-of-Thought
CPU	Central Processing Unit
GPU	Graphics Processing Unit
HPC	High-Performance Computing
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JSON-RPC	JavaScript Object Notation Remote Procedure Call
LLM	Large Language Model
MCP	Model Context Protocol
NLP	Natural Language Processing
RAG	Retrieval-Augmented Generation
ReAct	Reasoning and Acting
REST	Representational State Transfer
RLHF	Reinforcement Learning from Human Feedback
SDK	Software Development Kit
SQL	Structured Query Language
Slurm	Simple Linux Utility for Resource Management
SSE	Server-Sent Events
TLS	Transport Layer Security

Abbreviation	Description
UI	User Interface
URI	Uniform Resource Identifier

Contents

Abstract	iv
List of Abbreviations	vi
1 Introduction	1
1.1 Motivation	1
1.2 Scope and Goals	3
1.2.1 Project Goals	3
1.2.2 Project Scope	4
1.3 Challenges and Solutions	4
1.3.1 Challenge 1: Bridging Natural Language to System Commands	5
1.3.2 Challenge 2: Managing Complex Multi-Step Operations	5
1.3.3 Challenge 3: Real-Time Response Delivery	5
1.3.4 Challenge 4: Secure Tool Execution	6
1.3.5 Challenge 5: Visualization Generation and Delivery	6
1.4 Thesis Structure	7
2 Theoretical Background	9
2.1 Slurm Workload Manager	9
2.1.1 Architecture Overview	9
2.1.2 Core Concepts	10
2.1.3 Command-Line Interface	11
2.1.4 Job Submission and Scheduling	12
2.2 Text-to-SQL Concept	13
2.2.1 Problem Formulation	13

2.2.2	Typical Text-to-SQL Pipeline	13
2.2.3	Core Challenges	14
2.2.4	Relevance to This Project	14
3	Related Tools and Works	15
3.1	Related Works	15
3.1.1	HPC Management Interfaces	15
3.1.2	LLM Agent Frameworks	15
3.1.3	MCP Ecosystem and Relevance	16
3.1.4	Positioning of This Work	16
4	Proposed Approaches	17
4.1	Application Introduction	17
4.1.1	Application Concept	17
4.1.2	Target Users	18
4.1.3	Interaction Model	19
4.2	System Architecture	20
4.2.1	Architecture Overview	20
4.2.2	Component Descriptions	20
4.2.3	Communication Patterns	23
4.2.4	Data Flow	23
4.2.5	Deployment Architecture	24
5	Implementation	25
5.1	Agent Backend Implementation	25
5.1.1	Technology Stack	25
5.1.2	Runtime Agent Topology	26
5.1.3	Safety and Confirmation Pipeline	26
5.1.4	Session Management	26
5.1.5	Streaming Protocol (Current Format)	27
5.1.6	Chart Artifact Isolation	27
5.1.7	Evaluation-Oriented Backend Hooks	28
5.1.8	TODO for Final Thesis Freeze	28

5.2	MCP Server Implementation	28
5.2.1	Technology Stack	29
5.2.2	Architectural Design Decisions	29
5.2.3	Tool Implementation Patterns	30
5.2.4	Command Execution Infrastructure	36
5.2.5	Transport Configuration	36
5.2.6	Security Considerations	37
5.3	Frontend Implementation	38
5.3.1	Architecture Overview	38
5.3.2	Technology Stack	38
5.3.3	Primary Frontend Structure (React Application)	39
5.3.4	Streaming Response Flow	39
5.3.5	Evaluation UI Integration	40
5.3.6	Optional Open WebUI Integration Path	40
5.3.7	Agent Service API	40
5.3.8	Current User-Facing Capabilities	41
5.3.9	Deployment Configuration	41
5.3.10	TODO for Final Thesis Freeze	41
6	Experiments	43
6.1	Testing Approach	43
6.1.1	Testing Environment	43
6.1.2	Scenario-Grid Dataset	45
6.1.3	Automated Evaluation Framework	46
6.1.4	Manual Scenario Validation	48
6.1.5	Running the Evaluation	48
6.1.6	Source Code and Reproducibility	48
6.1.7	TODO for Final Thesis Freeze	49
6.2	Experimental Results	49
6.2.1	Benchmark Configuration Snapshot	49
6.2.2	Overall Performance	50
6.2.3	Component-Level Averages	50

6.2.4	Results by Category	51
6.2.5	Results by Scenario	51
6.2.6	Failure Pattern Analysis	52
6.2.7	Latency Characteristics	52
6.2.8	Manual Scenario Validation Summary	52
6.2.9	Comparison with Manual CLI Workflow	53
6.2.10	Result Interpretation	53
6.2.11	TODO for Final Thesis Freeze	53
7	Conclusion	55
7.1	Summary	55
7.1.1	Research Contributions	55
7.1.2	Achievement Against Project Goals	56
7.1.3	Key Quantitative Outcomes	56
7.1.4	Main Findings	57
7.1.5	Source Code Availability	57
7.1.6	TODO for Final Thesis Freeze	57
7.2	Limitations	57
7.2.1	Evaluation Realism Limitations	58
7.2.2	Behavioral Reliability Limitations	58
7.2.3	Operational Scope Limitations	58
7.2.4	Security and Governance Limitations	58
7.2.5	Performance and Scalability Limitations	59
7.2.6	Model and Scoring Limitations	59
7.2.7	TODO for Limitation Mitigation	59
	References	59

List of Tables

- 5.1 MCP Server Tool Reference 31
- 5.2 Predefined Analysis Scripts 33
- 5.3 Predefined Chart Visualizations 34

- 6.1 Test Environment Specifications 44
- 6.2 Mock Data Scenarios 45
- 6.3 Current Dataset Distribution (367 Cases) 46
- 6.4 Evaluation Run Configuration (Latest Snapshot) 49
- 6.5 Overall Metrics (367-Case Snapshot) 50
- 6.6 Average Component Scores 50
- 6.7 Category-Level Results (Latest Snapshot) 51
- 6.8 Scenario-Level Robustness 51
- 6.9 Mismatch Counts Across 367 Cases 52
- 6.10 Task Complexity: Agent vs Manual CLI 53

- 7.1 Final Snapshot Highlights 56

List of Figures

2.1	Slurm System Architecture	10
4.1	Slurm Agent Application Concept	18
4.2	Slurm Agent Flow Chart	19
4.3	Slurm Agent System Architecture	20
5.1	Confirmation Flow for Dangerous Operations	26
5.2	Chart Artifact Isolation in the Backend Pipeline	28
5.3	Frontend Integration Architecture	38
5.4	Frontend Demo Interface	41
6.1	Testing Environment Architecture	46

Chapter 1

Introduction

1.1 Motivation

High-Performance Computing (HPC) has become an essential infrastructure in modern scientific research, engineering simulations, and large-scale data processing. HPC clusters enable researchers to tackle computationally intensive problems that would be infeasible on conventional computing systems. From climate modeling and genomics to machine learning and financial simulations, HPC systems provide the computational power necessary to drive innovation across multiple domains.

The Simple Linux Utility for Resource Management (Slurm) has emerged as the dominant workload manager for HPC clusters, managing resources in approximately 60% of the world's top supercomputers [24, 15]. Slurm provides sophisticated mechanisms for job scheduling, resource allocation, and cluster monitoring, enabling efficient utilization of computational resources across thousands of nodes. The system's flexibility and scalability have made it the preferred choice for both academic institutions and commercial organizations operating large-scale computing facilities.

Despite its power and flexibility, Slurm presents significant usability challenges that create barriers for both novice and experienced users [14]. The system exposes its functionality through a complex command-line interface (CLI) comprising numerous commands, each with extensive sets of options and parameters. Users must master commands such as `squeue`, `sinfo`, `sbatch`, `scontrol`, and `sacct`, along with their associated flags and output formats. This complexity is compounded by the need to write job submis-

sion scripts in Bash, which requires additional technical knowledge beyond the domain expertise of many researchers.

The learning curve associated with Slurm administration and usage creates several operational challenges. System administrators spend considerable time training new users, while researchers face delays in their work as they navigate the intricacies of job submission and monitoring. Common tasks such as checking job status, understanding resource availability, or debugging failed jobs often require consulting extensive documentation or seeking expert assistance.

Recent advances in Large Language Models (LLMs) have demonstrated remarkable capabilities in natural language understanding and generation [4, 10]. These models can interpret complex user queries, reason about multi-step tasks, and generate appropriate responses or actions. The emergence of LLM-based agents—systems that combine language models with tool-calling capabilities—has opened new possibilities for creating intelligent interfaces to complex systems [22, 16].

The Model Context Protocol (MCP), developed by Anthropic, provides a standardized approach for connecting LLMs to external tools and data sources [1]. MCP enables the creation of modular, reusable integrations that can expose system functionality to AI agents in a secure and controlled manner. This architectural pattern separates the concerns of tool implementation from agent logic, facilitating the development of sophisticated AI-assisted applications.

This research is motivated by the opportunity to bridge the gap between Slurm’s powerful capabilities and user accessibility through an intelligent conversational interface. By leveraging LLM agents connected to Slurm through MCP, we can create a system that allows users to manage cluster resources using natural language queries, eliminating the need to memorize complex command syntax and reducing the cognitive load associated with HPC operations.

1.2 Scope and Goals

1.2.1 Project Goals

The primary objective of this research is to design and implement an intelligent agent system that provides a natural language interface for Slurm cluster management. This system aims to democratize access to HPC resources by enabling users to interact with Slurm using conversational queries rather than complex command-line syntax. The following goals guide the development of this solution:

Goal 1: Natural Language Interface for Slurm Operations. The system shall enable users to perform common Slurm operations through natural language queries. Users should be able to check job status, view cluster resources, submit jobs, and monitor queues without explicit knowledge of Slurm command syntax. The agent must interpret user intent accurately and translate queries into appropriate Slurm commands.

Goal 2: Multi-Agent Architecture with Specialized Capabilities. The system shall implement a multi-agent architecture where specialized agents handle different aspects of cluster management. This includes agents for command execution, data visualization, and web search. The handoff mechanism between agents should be seamless and transparent to the user.

Goal 3: Standardized Tool Integration via MCP. The system shall utilize the Model Context Protocol to provide standardized tool interfaces between the LLM agents and Slurm infrastructure. The MCP server should expose Slurm functionality as discrete tools that can be invoked by the agent, ensuring clean separation between the AI layer and system operations.

Goal 4: Visualization of Cluster Metrics. The system shall provide visualization capabilities for cluster data, enabling users to request charts and graphs showing resource utilization, job distributions, and other metrics. Visualizations should be generated dynamically based on user queries and current cluster state.

Goal 5: Real-Time Streaming Responses. The system shall support real-time streaming of agent responses using Server-Sent Events (SSE), providing users with immediate feedback during long-running operations. This improves user experience by eliminating

the perception of system unresponsiveness.

Goal 6: Web-Based User Interface. The system shall provide an intuitive web-based interface that supports markdown rendering, code highlighting, and inline visualization display. The interface should be accessible and usable without specialized client software.

1.2.2 Project Scope

The scope of this project encompasses the design, implementation, and evaluation of the Slurm Agent system. The following boundaries define what is included and excluded from the project:

In Scope:

- Development of an MCP server exposing Slurm functionality as tools
- Implementation of a multi-agent system using the OpenAI Agents SDK
- Design and implementation of a web-based frontend interface
- Integration of chart visualization capabilities using matplotlib
- Implementation of web search functionality for documentation assistance
- Real-time response streaming using SSE
- Testing on a single-node Slurm cluster configuration

This project focuses on demonstrating the feasibility and effectiveness of using LLM agents with MCP for HPC cluster management, providing a foundation for future research and development in this domain.

1.3 Challenges and Solutions

The development of an LLM-based agent for Slurm cluster management presents several technical and architectural challenges. This section identifies these challenges and outlines the approaches adopted to address them.

1.3.1 Challenge 1: Bridging Natural Language to System Commands

Challenge Description: Slurm operations require precise command syntax with specific parameters and flags. Natural language queries are inherently ambiguous and may not provide all necessary information for command execution. The system must accurately interpret user intent and request clarification when required, while avoiding incorrect command execution that could impact cluster operations.

Solution Approach: We employ a ReAct (Reasoning and Acting) architecture [23] where the main agent follows a Think $\rightarrow$ Act $\rightarrow$ Observe loop. The MCP server exposes Slurm commands as tools with well-defined Pydantic parameter schemas, enabling the LLM to understand required and optional parameters. The agent reasons about each query before acting, and can engage in multi-turn dialogue using SQLite-backed conversation history to gather missing information from users.

1.3.2 Challenge 2: Managing Complex Multi-Step Operations

Challenge Description: Many cluster management tasks require multiple sequential operations with dependencies between steps. For example, analyzing job performance may require first retrieving job information, then querying accounting data, and finally generating a visualization. Managing these workflows while maintaining coherent conversation context presents significant complexity.

Solution Approach: We implement a multi-agent architecture where the main ReAct agent orchestrates specialized sub-agents invoked as tools. An Analysis Sub-Agent handles read-only queries (cluster status, job information, web search), while an Action Sub-Agent manages job operations. Sub-agents return results to the main agent, which maintains overall context and coordinates execution. A `ChartFilteredSession` wrapper strips Mermaid chart code from conversation history before sending to the LLM, reducing token consumption since visualization code is verbose but not needed for reasoning.

1.3.3 Challenge 3: Real-Time Response Delivery

Challenge Description: LLM inference and Slurm command execution can introduce noticeable latency, potentially leading to poor user experience. Users may perceive the

system as unresponsive during processing, particularly for complex queries that require multiple tool invocations or lengthy computations.

Solution Approach: We implement Server-Sent Events (SSE) with structured delta fields so the frontend can render partial results incrementally. The stream separates status updates, tool output, reasoning text, final content, chart artifacts, and pending-action metadata. This design powers the primary React frontend and remains compatible with optional middleware/OpenWebUI deployment paths.

1.3.4 Challenge 4: Secure Tool Execution

Challenge Description: The system must execute Slurm commands on behalf of users, raising concerns about command injection, unauthorized operations, and potential system impact. Improper handling could allow malicious queries to execute harmful commands on the cluster.

Solution Approach: The system implements a layered security model. First, tools are categorized into safe read-only operations (squeue, sinfo, sacct) and dangerous write operations (scancel, sbatch, scontrol updates). Safe tools execute immediately, while dangerous tools are queued in a persistent Pending Actions Store requiring explicit user confirmation. The MCP server uses parameterized tool interfaces with Pydantic validation that prevent arbitrary command injection. Dynamic tool filters ensure sub-agents can only access their permitted tool sets.

1.3.5 Challenge 5: Visualization Generation and Delivery

Challenge Description: Generating and delivering data visualizations within a conversational interface presents unique challenges. The system must determine appropriate chart types based on data characteristics, generate visualizations dynamically, and deliver them through the SSE response stream in a format consumable by the web frontend.

Solution Approach: A dedicated chart generation tool handles visualization using Mermaid diagram syntax. The main agent calls the `generate_chart` tool via the MCP server, which returns Mermaid code for chart types including pie charts, bar charts, and flowcharts. Chart artifacts are stored in a context object during the run and transmitted through the SSE stream as Mermaid code blocks. The frontend renders these using

a Mermaid renderer, providing inline visualization within the conversation. Critically, a `ChartFilteredSession` wrapper makes the agent “blind” to chart artifacts—Mermaid blocks are stripped from conversation history before LLM inference, significantly reducing token consumption since chart code can be hundreds of tokens that provide no reasoning value.

1.4 Thesis Structure

This thesis is organized into seven chapters, each addressing specific aspects of the research and development of the Slurm Agent system. The structure is designed to provide a logical progression from theoretical foundations through implementation details to experimental evaluation.

Chapter 1: Introduction. This chapter presents the motivation for developing an LLM-based agent for Slurm cluster management. It establishes the scope and goals of the project, identifies key challenges encountered during development, and outlines the solutions adopted to address them.

Chapter 2: Theoretical Background. This chapter provides the foundational knowledge necessary to understand the research. It covers Slurm workload management, including architecture and key concepts; Large Language Models and their capabilities; and the Model Context Protocol for tool integration. These concepts form the technical basis upon which the solution is built.

Chapter 3: Related Tools and Works. This chapter surveys existing solutions and related research in the domains of HPC management interfaces, LLM-based agent frameworks, and MCP implementations. It analyzes the strengths and limitations of current approaches, positioning the contribution of this research within the broader landscape.

Chapter 4: Proposed Approaches. This chapter presents the design of the Slurm Agent system. It begins with an introduction to the application concept, followed by detailed requirements analysis. The chapter describes the overall system architecture and the multi-agent design that enables specialized handling of different task types.

Chapter 5: Implementation. This chapter details the implementation of each system component. It covers the MCP server development for exposing Slurm functionality,

the agent backend implementation using the OpenAI Agents SDK, and the frontend development including the SSE-based real-time update mechanism and visualization rendering.

Chapter 6: Experiments. This chapter describes the testing methodology and presents experimental results. It covers the testing approaches employed, the test environment configuration, and evaluation of the system’s capabilities across various usage scenarios.

Chapter 7: Conclusion. This chapter summarizes the research contributions and findings. It discusses limitations of the current implementation and identifies opportunities for future work and potential extensions of the system.

Chapter 2

Theoretical Background

2.1 Slurm Workload Manager

Slurm (Simple Linux Utility for Resource Management) is an open-source, highly scalable cluster management and job scheduling system designed for Linux clusters [24]. Originally developed at Lawrence Livermore National Laboratory, Slurm has become the predominant workload manager in the HPC industry, deployed on approximately 60% of the world's top 500 supercomputers. This section provides an overview of Slurm's architecture, core concepts, and operational mechanisms.

2.1.1 Architecture Overview

Slurm employs a centralized controller architecture with distributed compute node daemons. The system consists of three primary daemon components that work together to manage cluster resources and execute user workloads.

The **slurmctld** daemon serves as the central management controller, running on a dedicated management node. This daemon maintains the global state of the cluster, including information about nodes, partitions, jobs, and resource allocations. The controller is responsible for accepting job submissions, making scheduling decisions, and dispatching work to compute nodes. For high availability, Slurm supports backup controller configurations where a secondary **slurmctld** can assume control if the primary fails.

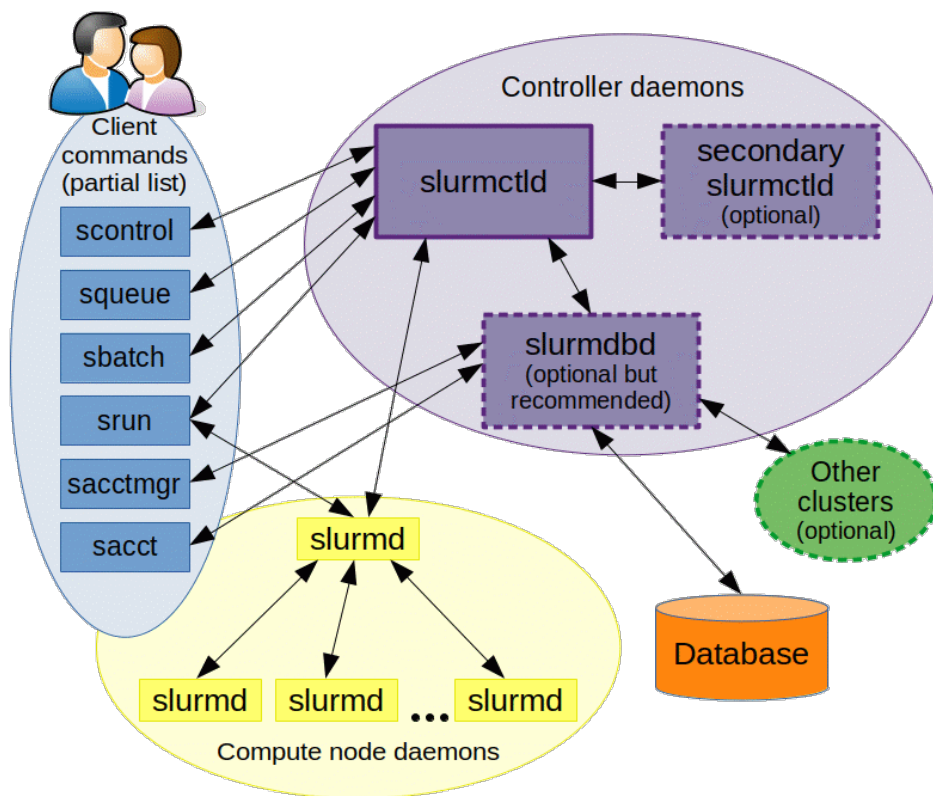


Figure 2.1: Slurm System Architecture

The **slurmd** daemon runs on each compute node in the cluster, acting as a remote execution agent. Each slurmd is responsible for launching, monitoring, and terminating jobs on its local node. The daemon reports node state and resource availability to the central controller and receives job execution requests. Slurmd ensures proper resource isolation for running jobs and handles signal propagation for job control.

The **slurmdbd** daemon (Slurm Database Daemon) provides an optional but commonly used accounting interface. This daemon manages connections to a backend database (typically MySQL or MariaDB) where job accounting records, resource usage statistics, and user associations are stored. The database daemon enables administrators to track resource consumption, generate usage reports, and implement fair-share scheduling policies.

2.1.2 Core Concepts

Understanding Slurm requires familiarity with several fundamental concepts that define how resources are organized and allocated.

Nodes represent individual compute servers in the cluster. Each node has defined resources including CPUs (or cores), memory, GPUs, and potentially other generic resources. Nodes can be in various states: idle (available), allocated (running jobs), down (unavailable), or drained (not accepting new work). Node features and constraints allow jobs to request specific hardware characteristics.

Partitions serve as logical groupings of nodes, functioning similarly to queues in other scheduling systems. Partitions define access controls, resource limits, and scheduling policies for different user groups or job types. A single node can belong to multiple partitions, and users submit jobs to specific partitions based on their requirements and permissions.

Jobs are resource allocation requests submitted by users. A job specification includes resource requirements (nodes, CPUs, memory, time limit), the program to execute, input/output file locations, and various optional parameters. Jobs progress through states including pending, running, completing, and completed (or failed/cancelled).

Steps are discrete execution units within a job. While a job represents the overall resource allocation, steps allow multiple parallel or sequential programs to execute within that allocation. Each step can use a subset of the job's allocated resources.

2.1.3 Command-Line Interface

Slurm provides a comprehensive suite of command-line utilities for cluster interaction:

- `sinfo`: Displays information about nodes and partitions, including their state, resources, and availability
- `squeue`: Shows the queue of pending and running jobs, with details about resource allocation and status
- `sbatch`: Submits batch job scripts for later execution
- `srun`: Launches parallel jobs, either as standalone commands or within existing allocations
- `salloc`: Allocates resources for interactive use
- `scancel`: Cancels pending or running jobs

- `scontrol`: Administrative command for querying and modifying cluster configuration
- `sacct`: Retrieves accounting data for completed jobs

Each command supports numerous options and output format specifications, contributing to the system's flexibility but also its complexity. Understanding the full capabilities of these tools requires significant learning investment, motivating the development of more accessible interfaces.

2.1.4 Job Submission and Scheduling

Job submission in Slurm typically involves creating a batch script that specifies resource requirements and execution commands. The script uses special comment directives (prefixed with `#SBATCH`) to declare parameters such as partition, time limit, node count, and resource constraints.

Slurm's scheduler evaluates pending jobs and available resources to make allocation decisions. The default scheduler uses a priority-based algorithm with backfill optimization, where smaller jobs can be scheduled in gaps left by larger pending jobs. Administrators can configure various priority factors including job age, fair-share usage, job size, and quality-of-service levels.

The scheduling system supports advanced features including:

- Job arrays for submitting multiple similar jobs efficiently
- Job dependencies for defining execution order relationships
- Preemption policies for priority-based job interruption
- Resource reservations for guaranteed future allocations

These capabilities make Slurm suitable for complex workflows but increase the knowledge required for effective utilization.

2.2 Text-to-SQL Concept

Text-to-SQL is the task of translating natural language questions into executable SQL queries over a given database schema. In recent years, it has become a representative benchmark for grounded semantic parsing because the output is formally structured, directly executable, and easy to verify by execution results [27, 25].

2.2.1 Problem Formulation

Given a user question q and schema context S (tables, columns, and relations), the model predicts an SQL query y :

$$\hat{y} = \arg \max_y P(y \mid q, S) \quad (2.1)$$

Unlike free-form text generation, valid output must satisfy SQL grammar and schema constraints. Therefore, many systems combine language understanding with structural constraints during decoding.

2.2.2 Typical Text-to-SQL Pipeline

A practical Text-to-SQL pipeline usually contains four steps:

- **Schema linking:** map question mentions (e.g., “running jobs”, “gpu partition”) to schema elements.
- **Intent and operator inference:** determine filters, aggregations, joins, ordering, and grouping.
- **Constrained decoding:** generate SQL under grammar/schema constraints.
- **Execution validation:** run query and check whether output is plausible and non-empty when expected.

Modern approaches improve this process with relation-aware schema encoding and stronger schema-question alignment [19].

2.2.3 Core Challenges

Text-to-SQL remains difficult due to:

- **Ambiguity**: natural language can under-specify filters or aggregation intent.
- **Schema scale**: large schemas increase linking and join complexity.
- **Compositional queries**: nested logic and multi-table reasoning are error-prone.
- **Execution risk**: syntactically valid SQL may still be semantically wrong.

For this reason, evaluation usually combines exact-match metrics with execution-based correctness.

2.2.4 Relevance to This Project

Although the Slurm Agent does not directly generate SQL for end users, Text-to-SQL provides an important conceptual analogue: **mapping natural language into constrained, executable system operations**. The same principles are reused here:

- structured grounding against an explicit tool/schema interface,
- constrained action generation,
- execution-time validation and safety checks.

In this thesis, these ideas appear as tool-schema grounding, routing constraints, and human-in-the-loop confirmation for dangerous operations.

Chapter 3

Related Tools and Works

3.1 Related Works

This section summarizes the most relevant prior work categories that informed this project, with focus on practical comparison rather than exhaustive survey detail.

3.1.1 HPC Management Interfaces

Traditional Slurm interaction remains command-line centric (e.g., `squeue`, `sacct`, `sinfo`, `sbatch`) and highly expressive, but it requires substantial syntax knowledge and operational experience. Existing web portals improve accessibility for standard workflows, yet they still depend on users understanding scheduler concepts and parameter structures.

The gap addressed by this project is not raw command coverage, but intent translation: converting natural-language operational requests into safe, structured scheduler interactions.

3.1.2 LLM Agent Frameworks

Recent agent frameworks demonstrate that tool-augmented LLMs can perform multi-step reasoning and external action execution. Patterns such as ReAct and multi-agent delegation are particularly relevant to operational domains where a single response may require multiple tool calls and intermediate state checks.

For HPC usage, this motivates splitting responsibilities between read-oriented diagnosis and mutation-oriented operation execution, then evaluating both through architecture-aware metrics rather than text-only quality.

3.1.3 MCP Ecosystem and Relevance

The Model Context Protocol provides a standard interface layer between agent logic and external tools. Prior MCP work shows clear benefits for modularity and portability: tool servers can evolve independently of agent prompts and runtime orchestration.

In this project, MCP is used to expose Slurm-oriented tools and deterministic scenario resets for reproducible evaluation. The practical contribution is an HPC-focused, state-aware MCP usage pattern integrated with a streaming, multi-agent runtime.

3.1.4 Positioning of This Work

Compared with existing interfaces and generic agent demos, this project contributes a tighter integration of:

- natural-language Slurm assistance,
- explicit HITL safeguards for dangerous actions,
- state-resettable scenario-grid evaluation,
- frontend-integrated benchmarking workflow.

This positioning emphasizes reliable operational behavior under repeatable test conditions, which is critical for HPC assistant credibility.

Chapter 4

Proposed Approaches

4.1 Application Introduction

The Slurm Agent is an intelligent conversational interface designed to democratize access to High-Performance Computing cluster management. By combining Large Language Models with the Model Context Protocol, the system enables users to interact with Slurm workload managers through natural language queries, eliminating the need to memorize complex command syntax and reducing the cognitive overhead associated with HPC operations.

4.1.1 Application Concept

The fundamental concept behind the Slurm Agent is to create an intermediary layer between users and the Slurm cluster that understands natural language, reasons about appropriate actions, and presents results in accessible formats. Users express their needs conversationally—such as “What jobs are running on the GPU partition?” or “Show me a chart of my job history”—and the agent translates these queries into appropriate Slurm commands, executes them, and presents formatted results.

The system implements a **ReAct (Reasoning and Acting)** architecture pattern, where the main agent follows a Think → Act → Observe loop to process user requests. This enables self-correction on failures and iterative refinement of responses based on tool execution results.

[Placeholder: Updated high-level concept diagram showing user, chat UI, multi-agent backend, MCP server, and Slurm cluster.]

Figure 4.1: Slurm Agent Application Concept

The application provides several key capabilities:

Natural language query processing: Users can ask questions about cluster state, job status, and resource availability using conversational language. The main agent interprets intent and delegates to specialized sub-agents for data gathering.

Multi-agent orchestration: A main ReAct agent orchestrates specialized sub-agents—an Analysis Sub-Agent for read-only queries (cluster status, job information, web search) and an Action Sub-Agent for job management operations (submit, cancel, hold, release).

Command execution with safety controls: The system distinguishes between safe read-only operations and dangerous write operations. Dangerous actions (job cancellation, configuration changes) are queued and require explicit user confirmation before execution.

Data visualization: Cluster metrics and job statistics can be visualized as charts and graphs through a dedicated chart generation tool, providing intuitive understanding of complex data.

Documentation assistance: When users encounter errors or need guidance, the agent can search relevant documentation via web search integration and provide contextual help.

Real-time streaming: Responses stream to users in real-time, providing immediate feedback during processing and maintaining a responsive user experience.

4.1.2 Target Users

The application serves multiple user categories with different needs:

Researchers and scientists who use HPC resources for computational work but may lack deep system administration knowledge. These users benefit from the ability to check job status, understand resource availability, and troubleshoot issues without mastering Slurm’s command-line interface.

New HPC users face a steep learning curve when first accessing cluster resources.

The Slurm Agent reduces this barrier by enabling productive interaction from the start, with the agent handling translation to system operations.

System administrators can use the agent for rapid cluster status checks and routine operations, leveraging natural language for efficiency. The agent supplements rather than replaces direct command-line access.

Support staff helping users with HPC-related questions can use the agent to quickly gather diagnostic information and provide guided assistance.

4.1.3 Interaction Model

User interaction with the Slurm Agent follows a conversational paradigm. Users type messages in a chat interface, and the agent responds with text, formatted data, or visualizations. The conversation maintains context across multiple turns through persistent session management, allowing follow-up questions and clarifications.

A typical interaction flow includes:

1. User submits a natural language query through the web interface
2. Main ReAct agent reasons about the query (*Think* phase)
3. Agent determines necessary actions and invokes appropriate sub-agents (*Act* phase)
4. Sub-agents call tools to execute Slurm commands
5. Tool results are observed and processed (*Observe* phase)
6. For dangerous operations, action is queued pending user confirmation
7. Agent generates a natural language response with formatted results
8. Visualizations, if requested, are rendered inline as charts
9. User can continue with follow-up queries using preserved context

[Placeholder: Updated end-to-end interaction flowchart (user prompt, agent reasoning, tool calls, HITL confirm/cancel, streamed response).]

Figure 4.2: Slurm Agent Flow Chart

This model enables intuitive interaction while maintaining the full power of Slurm’s capabilities through the underlying tool integration, with safety controls preventing accidental cluster modifications.

4.2 System Architecture

This section presents the overall architecture of the Slurm Agent system, describing the major components, their interactions, and the rationale behind architectural decisions. The architecture follows a layered design that separates user interface, agent logic, and system integration concerns.

4.2.1 Architecture Overview

The Slurm Agent system consists of four main layers:

1. **Presentation Layer:** Web-based frontend for user interaction
2. **Agent Layer:** Multi-agent backend handling reasoning and orchestration
3. **Protocol Layer:** MCP server providing standardized tool interfaces
4. **Infrastructure Layer:** Slurm cluster and external services

[Placeholder: Updated system architecture diagram with presentation, agent, protocol, and infrastructure layers plus data/control flow.]

Figure 4.3: Slurm Agent System Architecture

4.2.2 Component Descriptions

Presentation Layer

The Presentation Layer provides the user-facing interface for interacting with the Slurm Agent. This layer is responsible for:

- Rendering a conversational chat interface with message history

- Capturing user queries in natural language
- Displaying streaming responses in real-time
- Rendering formatted content including markdown, code blocks, and inline visualizations
- Managing confirmation workflows for dangerous operations
- Preserving session state across interactions

The interface follows a conversational paradigm where users type queries and receive responses in a familiar messaging format. This design abstracts the complexity of Slurm commands behind natural language interaction.

Agent Layer

The Agent Layer implements the core reasoning and orchestration logic using a multi-agent architecture. The system employs a hierarchical structure with specialized agents:

- **Main ReAct Agent:** The primary orchestrator that implements the Think → Act → Observe reasoning loop. This agent maintains conversation context, interprets user intent, and delegates to specialized sub-agents based on the nature of the request.
- **Analysis Sub-Agent:** Handles read-only information gathering operations including cluster status queries, job queue inspection, resource monitoring, and documentation search. This agent can chain multiple tool calls to gather comprehensive information.
- **Action Sub-Agent:** Manages operations that modify cluster state, such as job submission, cancellation, and configuration changes. This agent enforces safety controls and confirmation requirements for dangerous operations.

The Main Agent also has direct access to auxiliary tools including chart generation for data visualization, which calls the Protocol Layer to render cluster metrics and statistics as visual charts.

Key architectural features of the Agent Layer include:

- **Session Persistence:** Conversation history is maintained across interactions, enabling contextual follow-up queries
- **Confirmation Workflow:** Dangerous operations are queued pending explicit user confirmation before execution
- **Token Optimization:** Visualization artifacts are filtered from conversation history to reduce token consumption
- **Streaming Output:** Responses are streamed incrementally to provide immediate user feedback

Protocol Layer

The Protocol Layer implements the Model Context Protocol (MCP) to provide a standardized interface between the Agent Layer and the underlying Slurm infrastructure. This layer:

- **Exposes Slurm Operations as Tools:** Each Slurm command is wrapped as a discrete tool with defined parameters and return types
- **Validates Parameters:** Input validation prevents malformed or dangerous command construction
- **Executes Commands Safely:** Commands are executed with appropriate isolation and error handling
- **Formats Output:** Raw command output is processed into structured formats suitable for agent consumption

The MCP abstraction provides several benefits:

- Decouples agent logic from Slurm-specific implementation details
- Enables tool discovery and introspection by the agent
- Supports extension with new tools without modifying the agent core
- Provides a security boundary for command execution

Infrastructure Layer

The Infrastructure Layer comprises the external systems that the agent interacts with:

- **Large Language Model:** Provides the reasoning capabilities for natural language understanding, intent interpretation, and response generation (either local or cloud API)
- **Slurm Cluster:** The target HPC infrastructure being managed, accessed through standard Slurm commands
- **Web Search Service:** Enables retrieval of documentation and troubleshooting information from external sources
- **Context Database:** SQLite database for storing session state and pending confirmation actions

4.2.3 Communication Patterns

The architecture employs several communication patterns:

Request-Response: Standard requests for initiating conversations and submitting queries.

Streaming: Unidirectional streaming from backend to frontend for real-time response delivery. The backend sends a continuous stream of events that the frontend processes incrementally.

Tool Invocation: Structured communication between the Agent Layer and Protocol Layer for tool discovery and execution.

4.2.4 Data Flow

A typical query flows through the system layers as follows:

1. User submits natural language query through the Presentation Layer
2. Agent Layer receives query and begins Think-Act-Observe reasoning
3. Agent delegates to appropriate sub-agent based on query type

4. Sub-agent invokes tools via Protocol Layer to gather data or perform actions
5. Results propagate back through the layers
6. Agent synthesizes response and streams to user

For dangerous operations, an additional confirmation step interrupts the flow: the pending action is stored and presented to the user, and execution proceeds only after explicit confirmation.

4.2.5 Deployment Architecture

The system components can be deployed in various configurations:

Local Development: All components run on a single machine with local Slurm installation or connection to a remote cluster.

Distributed Deployment: Frontend, backend, and MCP server run on separate hosts, with the MCP server deployed on a Slurm management node.

The modular architecture supports flexible deployment topologies based on infrastructure requirements and security considerations.

Chapter 5

Implementation

5.1 Agent Backend Implementation

This section documents the current backend implementation used in the updated project baseline (April 2026). The backend is responsible for: (i) maintaining session-aware multi-agent execution, (ii) exposing an OpenAI-compatible chat API for the frontend, and (iii) enforcing safety and confirmation policies for state-changing Slurm operations.

5.1.1 Technology Stack

The backend is implemented in Python and organized around asynchronous services:

- **FastAPI + Uvicorn:** API layer for chat, health, and session endpoints
- **OpenAI Agents SDK:** multi-agent orchestration and tool-driven execution
- **aiohttp/httpx:** asynchronous streaming and external HTTP calls
- **SQLiteSession:** persistent per-session context and chat history
- **MCP over SSE:** tool discovery and execution against the Slurm MCP server

Model routing is configurable at runtime (local Ollama and optional remote providers), while the default deployment currently uses local Ollama.

5.1.2 Runtime Agent Topology

The operational agent topology is a main orchestrator with two specialized sub-agent tools:

- `analyze_cluster`: read-only cluster inspection and diagnostics
- `manage_jobs`: state-changing operations (cancel, hold, release, submit, update)

In addition, the main agent directly exposes utility tools: `generate_chart`, `confirm_action`, `cancel_action`, and `check_pending_actions`.

This design keeps user-facing reasoning centralized while restricting high-risk execution to explicit action pathways.

5.1.3 Safety and Confirmation Pipeline

Dangerous operations are guarded through a queued-action workflow:

1. A dangerous request is transformed into a pending action (not executed immediately).
2. The user is shown the pending action and must explicitly confirm or cancel.
3. Execution occurs only through `confirm_action`; cancellation is handled by `cancel_action`.

This creates a two-step control path for destructive commands and reduces accidental cluster mutations.

[Placeholder: Updated confirmation workflow for dangerous operations (request parsing, pending action creation, user confirm/cancel, guarded execution).]

Figure 5.1: Confirmation Flow for Dangerous Operations

5.1.4 Session Management

The API maps each chat ID to a persistent agent session, enabling stateful multi-turn behavior. Session state includes:

- conversation memory,
- pending actions,
- chart artifact context,
- model/tool runtime configuration.

Session cleanup and explicit clear endpoints are provided to avoid stale context and to support evaluation reproducibility.

5.1.5 Streaming Protocol (Current Format)

Streaming responses use OpenAI-style SSE chunks with structured delta fields. Unlike earlier marker-based approaches, the current stream sends typed fields directly:

- `status_update`: tool progress and handoff status
- `tool_output`: raw tool output fragments
- `reasoning_content` / `reasoning`: model thinking tokens
- `content`: final user-visible answer tokens
- `chart_artifact`: Mermaid payloads for rendering
- `todo_update`: task-plan updates
- `pending_actions`: queued actions awaiting user confirmation

This explicit schema simplifies downstream parsing in both chat UI and evaluation pipelines.

5.1.6 Chart Artifact Isolation

Chart rendering uses Mermaid artifacts generated as tool outputs and streamed separately from natural-language reasoning. To prevent context pollution, chart artifacts are filtered out of future LLM context windows via session wrappers. This preserves token budget and avoids recursive visual-text contamination.

[Placeholder: Updated chart-artifact isolation flow (tool output extraction, artifact channeling, context filtering, frontend rendering).]

Figure 5.2: Chart Artifact Isolation in the Backend Pipeline

5.1.7 Evaluation-Oriented Backend Hooks

The backend now exposes behavior that supports architecture-level evaluation:

- deterministic session IDs for replayable traces,
- detailed status/tool streaming for trace extraction,
- compatibility with state-reset workflows in mock MCP mode,
- explicit HITL event visibility for safety scoring.

These hooks are consumed by the Scenario-Grid evaluation stack described in Chapter 6.1.

5.1.8 TODO for Final Thesis Freeze

- Export one final frozen prompt snapshot (main/analysis/action) into appendix for reproducibility.
- Add a compact "tool guardrail matrix" extracted from runtime configuration.
- Add sequence diagram for the exact SSE delta event flow (status/tool_output/reasoning/content)

5.2 MCP Server Implementation

This section details the implementation of the MCP server that exposes Slurm functionality as tools for the agent system. The server provides the bridge between the AI agents and the underlying cluster infrastructure, translating natural language-derived tool calls into actual Slurm command executions.

5.2.1 Technology Stack

The MCP server is implemented using Python 3.10+ as the primary language, selected for its compatibility with the MCP Python SDK and the agent backend. The official MCP Python SDK provides the protocol implementation and server infrastructure. Asynchronous I/O via the `asyncio` library enables non-blocking command execution, which is essential for maintaining responsiveness when multiple tool calls are in flight. Subprocess management handles the actual Slurm command invocation with proper output capture and error handling.

5.2.2 Architectural Design Decisions

Several key architectural decisions shape the MCP server implementation:

Stateless Tool Design

Each tool is implemented as a stateless function that takes parameters, executes a command, and returns results. This design simplifies scaling, testing, and error recovery. No session state is maintained between tool invocations, ensuring that each call is independent and idempotent for read operations.

Parameter Validation and Command Construction

Tool parameters are validated before command construction to prevent invalid Slurm command execution. Parameters are never directly interpolated into shell commands; instead, they are passed as discrete arguments to subprocess execution, preventing shell injection vulnerabilities. This parameterized approach ensures that user-provided values cannot alter command structure or execute unintended operations.

Output Formatting

The MCP server uses explicit format flags when invoking Slurm commands (e.g., `--format` for `squeue` and `sacct`) to ensure consistent, structured output suitable for LLM consumption. This approach produces pipe-delimited or tabular output that is predictable and easy to parse. Empty result sets are handled gracefully with appropriate header-only

responses, and error messages are converted to descriptive responses that the LLM can interpret and communicate to users.

5.2.3 Tool Implementation Patterns

Each Slurm operation is exposed as a distinct tool with well-defined parameters and behavior. Table 5.1 provides a complete reference of all available tools.

Table 5.1: MCP Server Tool Reference

Tool	Category	Description
<i>Query Tools (Read-Only)</i>		
squeue	Query	List jobs in the queue with filters
sacct	Query	Job accounting and history records
sinfo	Query	Node and partition status
scontrol_show	Query	Detailed job/node/partition info
sacctmgr_show	Query	Account/user/QOS information
sreport	Query	Usage reports and statistics
<i>Action Tools (State-Modifying)</i>		
sbatch	Submit	Submit batch job scripts
scancel	Control	Cancel running or pending jobs
scontrol_hold	Control	Hold pending jobs
scontrol_release	Control	Release held jobs
scontrol_update	Control	Modify job parameters
scontrol_requeue	Control	Requeue failed jobs
srund	Interactive	Run commands on allocated nodes
salloc	Interactive	Obtain interactive allocation
<i>Administrative Tools (High Risk)</i>		
scontrol_create	Admin	Create partitions/reservations
scontrol_delete	Admin	Delete partitions/reservations
scontrol_reconfigure	Admin	Reload Slurm configuration
sacctmgr_add	Admin	Add accounts/users/QOS
sacctmgr_modify	Admin	Modify account settings
sacctmgr_delete	Admin	Delete accounts/users
<i>Extended Tools</i>		
run_analysis	Script	Execute predefined analysis scripts
generate_chart	Visual	Generate data visualizations
web_search	Search	Search web for documentation/solutions

Query Tools

Query tools retrieve information from the Slurm cluster without modifying state. Each tool follows a consistent pattern: accept optional filter parameters, construct the appropriate Slurm command, execute asynchronously with timeout protection, and return formatted results. Error conditions return descriptive messages rather than raising exceptions, enabling the LLM to communicate issues to users.

Action Tools

Action tools modify cluster state through operations such as job submission, cancellation, hold, and release. These tools require additional safeguards because their effects are not reversible. The server implementation focuses on parameter validation and clear result reporting.

For dangerous operations (those that can cause data loss or significant disruption), the tools themselves do not implement confirmation logic. Instead, the agent backend layer intercepts these calls through the confirmation context mechanism described in Section 5.1. This separation of concerns keeps the MCP server focused on command execution while the agent layer handles safety policies.

Predefined Analysis Scripts

The `run_analysis` tool executes predefined shell scripts that perform comprehensive multi-step analysis operations. These scripts encapsulate common administrative workflows that would otherwise require multiple tool calls and complex reasoning. Table 5.2 lists the available scripts.

Table 5.2: Predefined Analysis Scripts

Script ID	Category	Description
<code>analyze_my_jobs</code>	Jobs	Comprehensive analysis of user's running, pending, completed, and failed jobs
<code>analyze_failed_jobs</code>	Debug	Deep analysis of failed, timed out, and OOM jobs with pattern identification
<code>analyze_pending_jobs</code>	Debug	Why jobs are pending—resource availability, queue position, wait estimates
<code>analyze_cluster_status</code>	Cluster	Complete cluster status—all partitions, nodes, utilization
<code>analyze_gpu_resources</code>	Cluster	GPU availability, allocation status, queue depth
<code>analyze_my_usage</code>	User	Usage statistics, resource consumption, fair share standing
<code>analyze_my_efficiency</code>	User	Job efficiency analysis—over-requesting resources?

Visualization Tools

The `generate_chart` tool creates data visualizations using predefined Python scripts that gather cluster data and render Mermaid diagrams. Each chart script is self-contained, executing necessary Slurm queries internally. Table 5.3 lists available visualizations.

Table 5.3: Predefined Chart Visualizations

Chart ID	Description
system_health	Dashboard showing node health, job counts, and resource utilization
cluster_topology	Visual representation of cluster structure and node states
pending_analysis	Analysis of pending jobs with resource requirements
resource_map	Resource allocation across partitions
job_lifecycle	Timeline visualization of job state transitions

Web Search Tool

The `web_search` tool enables the agent to retrieve external documentation, troubleshooting guides, and solutions for Slurm-related issues. This capability extends the agent’s knowledge beyond its training data, allowing it to provide up-to-date assistance.

Search Engine Integration The tool uses DuckDuckGo’s search API via the `duckduckgo-search` Python library, selected for its privacy-focused approach and lack of rate limiting compared to commercial alternatives. Search queries are passed directly to the API without modification—the implementation follows a “pass-through” design where user intent is preserved rather than enhanced or reformulated. The `region` parameter is set to `us-en` to ensure English-language results regardless of server location.

Content Fetching Beyond returning search snippets, the tool supports optional full-page content retrieval via the `fetch_content` parameter. When enabled, the tool fetches the HTML content of top results and extracts readable text by:

1. Removing script, style, navigation, and footer elements
2. Stripping HTML tags while preserving paragraph structure
3. Decoding HTML entities and normalizing whitespace
4. Truncating to a configurable maximum length (default: 3000 characters)

Bot Detection Bypass Many websites employ bot detection mechanisms that block requests from automated tools. The implementation uses a layered HTTP client strategy:

1. **Primary:** The `curl_cffi` library with browser impersonation (`chrome110` profile) provides TLS fingerprints identical to real browsers, bypassing fingerprint-based blocking used by sites like StackOverflow.
2. **Fallback:** The `httpx` library with browser-like headers (User-Agent, Accept-Language) handles sites without aggressive bot detection.

This approach successfully retrieves content from previously-blocked sources including StackOverflow, Reddit, and documentation sites that reject standard Python HTTP requests.

Parameters The web search tool accepts the following parameters:

- `query` (required): The search query string
- `max_results` (optional, default: 5): Maximum number of results to return
- `fetch_content` (optional, default: false): Whether to fetch full page content
- `fetch_top_n` (optional, default: 2): Number of top results to fetch content for

Limitations The web search implementation has several inherent limitations:

- **Region filtering unreliability:** DuckDuckGo's `region` parameter does not reliably filter results by language. The implementation includes a domain-based post-filter to exclude known non-English sites (e.g., `zhihu.com`, `baidu.com`), but this requires explicit domain enumeration and cannot catch all cases.
- **Bot detection:** Despite browser impersonation, some websites still block automated requests, resulting in failed content fetches for certain URLs.
- **Content extraction quality:** HTML-to-text extraction is heuristic-based and may lose important formatting, code blocks, or structural information from complex pages.

- **Rate limiting:** High-frequency searches may trigger DuckDuckGo's rate limiting, causing temporary failures.

5.2.4 Command Execution Infrastructure

Command execution is handled through a utility layer that provides consistent error handling, timeout management, and output processing.

Asynchronous Execution

Commands are executed asynchronously using Python's `asyncio` subprocess API. This non-blocking approach ensures that the MCP server remains responsive even when individual commands take significant time to complete. Multiple concurrent tool calls can be processed without blocking each other.

Timeout Management

All command executions are bounded by configurable timeouts (default: 30 seconds). This prevents the system from hanging indefinitely on unresponsive commands or deadlocked Slurm daemons. When a timeout occurs, the subprocess is terminated and an appropriate error message is returned to the caller.

Error Categorization

Command execution errors are categorized for appropriate handling: non-zero exit codes indicate Slurm-level errors (invalid job ID, permission denied) and return the `stderr` content as the error message; timeout errors indicate infrastructure issues and suggest retry or escalation; and exception errors indicate unexpected failures requiring investigation.

5.2.5 Transport Configuration

The server supports SSE (Server-Sent Events) transport for remote connections, enabling the agent backend to connect over HTTP. This transport choice reflects the deployment architecture where the MCP server may run on a different host than the agent backend,

particularly in scenarios where the MCP server runs directly on a cluster head node while the agent backend runs on a separate application server.

The SSE transport maintains a persistent connection over which tool calls and responses are transmitted as events. This eliminates the overhead of connection establishment for each tool call while providing reliable delivery through HTTP infrastructure.

5.2.6 Security Considerations

The MCP server implementation incorporates multiple security measures appropriate for HPC environment deployment.

Command Injection Prevention

As described above, all tool parameters are validated and passed as discrete subprocess arguments rather than interpolated into shell command strings. This fundamental design choice prevents command injection attacks regardless of parameter content.

Read-Focused Default Behavior

The primary tools focus on read operations that cannot cause harm. Write operations are explicitly categorized and can be restricted through configuration or agent-level policies. This defense-in-depth approach ensures that even if an attacker gains access to tool invocation, the damage potential is limited.

Output Sanitization

Command outputs are processed before returning to ensure that sensitive information is not inadvertently exposed. This includes filtering of internal paths, configuration details, and any data that should not be visible to end users.

Timeout Enforcement

Mandatory timeouts on all command executions prevent resource exhaustion attacks that could otherwise consume server resources indefinitely.

5.3 Frontend Implementation

This section describes the current frontend architecture for interacting with the Slurm Agent system. The active implementation is a custom React + Vite web application with two integrated modes: a chat interface and an evaluation dashboard. An Open WebUI bridge remains available as an optional compatibility deployment path.

5.3.1 Architecture Overview

The frontend stack can be used in two modes:

1. **Primary mode (used in this report):** React + Vite frontend (chat + evaluation views) communicating directly with the agent API and evaluation API.
2. **Compatibility mode:** Open WebUI + middleware router forwarding to the same agent API.

[Placeholder: Updated frontend integration architecture (React UI, agent API, evaluation API, SSE stream handling, optional Open WebUI bridge).]

Figure 5.3: Frontend Integration Architecture

5.3.2 Technology Stack

The active frontend uses the following stack:

- **React 18 + Vite:** client rendering and build tooling
- **FastAPI** [12]: agent backend and evaluation backend APIs
- **SSE over Fetch Streams:** incremental rendering of reasoning, status, and answer tokens
- **Mermaid:** inline chart rendering in chat/evaluation artifacts
- **LocalStorage-based session state:** frontend conversation persistence

5.3.3 Primary Frontend Structure (React Application)

The primary application is composed of modular React components:

- **App shell:** top-level layout and view switching (Chat/Eval)
- **Sidebar:** sessions, server settings, and mode toggle
- **Messages/Input components:** conversational interaction and HITL action buttons
- **Eval view:** dataset loading, run controls, live progress, per-test trace inspection

The integrated evaluation view allows end-to-end testing without leaving the primary UI. It displays per-test status, model traces, and aggregate metrics in real time.

5.3.4 Streaming Response Flow

The frontend consumes OpenAI-compatible SSE chunks from the backend. Key delta fields include:

- `status_update`
- `tool_output`
- `reasoning_content`
- `content`
- `chart_artifact`
- `todo_update`
- `pending_actions`

This structured stream enables independent rendering of tool traces, model thinking, final answer content, and user confirmation controls.

5.3.5 Evaluation UI Integration

The evaluation interface is now a first-class frontend mode. It provides:

- dataset load/upload and filtering,
- run-all and run-single controls,
- real-time test progression with pass/fail states,
- per-test drill-down for tools, routing, HITL, response, thinking, and judge output,
- aggregate metrics display.

This integration shortens the iteration loop between feature development and behavioral evaluation.

5.3.6 Optional Open WebUI Integration Path

Open WebUI integration remains available for environments requiring Open WebUI ecosystem features (organizational deployment, existing plugin stack). In this mode, a middleware router forwards chat requests to the same agent backend. This path is considered optional in the current implementation baseline.

5.3.7 Agent Service API

The frontend communicates with OpenAI-compatible endpoints exposed by `agent/main.py`. Key endpoints include:

- `GET /v1/models`: model list exposure
- `POST /v1/chat/completions`: The primary chat endpoint that processes user queries through the multi-agent system and streams responses
- `GET /health`: Health check endpoint for container orchestration

The dedicated evaluation backend (`evaluation/eval_server.py`) exposes additional endpoints for dataset loading, test execution, stopping, and persistent result snapshots.

5.3.8 Current User-Facing Capabilities

The frontend currently supports:

- live chat with streaming responses,
- HITL confirmation/cancellation controls,
- inline markdown and Mermaid rendering,
- multi-session navigation,
- integrated evaluation workflow in a dedicated tab.

Figure 5.4 shows the current web interface.

[Placeholder: Updated frontend screenshot showing chat mode, integrated Eval tab, live status stream, and HITL action controls.]

Figure 5.4: Frontend Demo Interface

5.3.9 Deployment Configuration

The project includes script-based startup for local development:

- **start.sh**: launches MCP server, agent API, and built frontend preview
- **start_eval_server.sh**: launches standalone evaluation backend
- **MCP scenario switching**: allows deterministic evaluation in healthy/failed/pending/mixed/debug_needed states

Containerized and Open WebUI-based deployments remain possible, but are not the only deployment model.

5.3.10 TODO for Final Thesis Freeze

- Add one updated screenshot of the integrated Eval tab (current figure set is chat-heavy).

- Add a sequence diagram for chat mode vs eval mode request paths.
- Document exact frontend build/version hash used for final experiments.

Chapter 6

Experiments

6.1 Testing Approach

This chapter describes the updated testing methodology used for the current Slurm Agent baseline. The evaluation focus has shifted from response-only quality checks to architecture-aware behavioral evaluation, including tool selection, routing behavior, HITL safety, and state-transition consistency.

6.1.1 Testing Environment

The project supports both real Slurm mode and mock mode. For reproducible benchmarking, the automated framework uses stateful mock scenarios exposed by the MCP server.

Hardware and Software Configuration

Table 6.1: Test Environment Specifications

Component	Specification
<i>Hardware</i>	
CPU	Intel Core i5-14600K (14 cores, 20 threads)
RAM	64 GB DDR5
GPU	NVIDIA GeForce RTX 5070 Ti (16 GB VRAM)
<i>Software</i>	
Operating System	Ubuntu 24.04 LTS (WSL2)
Python	3.11.13
Node.js	22.16.0
Ollama	0.12.3
<i>LLM Models</i>	
Main Agent (Reasoning)	Configurable (default local Ollama)
Sub-Agents (Tool Execution)	Configurable (same runtime provider)
LLM Judge (Evaluation)	qwen3.5:9b (current default)

The backend can route to multiple providers (local Ollama and optional remote providers). Final thesis measurements should pin one exact model/provider configuration for strict reproducibility.

Mock Data System

The evaluation framework resets mock state before each test case using source-state snapshots, then verifies behavior against expected target-state semantics. This design ensures deterministic test baselines while still exercising full agent-tool interactions.

Advantages include:

- **Reproducibility:** The same mock data produces consistent results across evaluation runs
- **Stateful Verification:** Expected source $\rightarrow$ target behavior is testable

- **Controlled Scenarios:** Multiple cluster conditions can be evaluated systematically
- **Architecture-Level Tracing:** tool calls, routing and HITL are all observable

Five scenarios are currently used:

Table 6.2: Mock Data Scenarios

Scenario	Description
healthy	Normal cluster operation with all jobs running correctly
failed	Cluster with multiple failed jobs including OOM, timeout, and script errors
pending	Heavy job queue with resources exhausted and jobs stuck in pending
mixed	Combination of running, pending, failed, and completed jobs
debug_needed	Complex error scenarios requiring web search to diagnose

6.1.2 Scenario-Grid Dataset

The dataset is generated by `evaluation/dataset.py` into `dataset.json`. Current dataset size is 367 test cases.

Table 6.3: Current Dataset Distribution (367 Cases)

Category	Count
read	106
diagnose	39
action	31
bulk	24
safety	45
submission	35
multi_step	17
account	25
edge	45
Total	367

Each test case includes:

- **input**: user prompt,
- **source_state**: pre-run mock jobs/nodes snapshot,
- **target_state**: expected post-action state,
- **ground_truth**: expected tools, handoff requirement, HITL requirement, and key response terms.

*[Placeholder: Updated testing architecture for scenario-grid evaluation
(frontend/backend/MCP/mock-state reset/judge pipeline).]*

Figure 6.1: Testing Environment Architecture

6.1.3 Automated Evaluation Framework

A dedicated evaluator (`scenario_eval.py`) and web evaluation backend (`eval_server.py`) execute tests against the live agent API while collecting architectural traces.

Test Case Structure

Each test evaluates architecture-level behavior, not just text quality:

- **Tool behavior:** expected and observed tools
- **Routing behavior:** Observer-only vs handoff to Operator
- **Safety behavior:** whether HITL was correctly triggered
- **State behavior:** action consistency with source/target state intent
- **Response behavior:** keyword/response coverage and optional LLM judge score

Scoring Methodology

When LLM judge is disabled, overall score is:

$$\text{overall} = 0.30 \cdot \text{tool_recall} + 0.25 \cdot \text{routing} + 0.25 \cdot \text{hitl} + 0.10 \cdot \text{keyword} + 0.10 \cdot \text{state} \quad (6.1)$$

When LLM judge is enabled, weights are adjusted and judge contributes 15%:

$$\text{overall}_{\text{judge}} = 0.25 \cdot \text{tool_recall} + 0.20 \cdot \text{routing} + 0.20 \cdot \text{hitl} + 0.10 \cdot \text{keyword} + 0.10 \cdot \text{state} + 0.15 \cdot \text{judge} \quad (6.2)$$

Pass condition uses threshold ≥ 0.80 with an additional keyword gate for response fidelity.

Aggregate Metrics

Beyond per-test scores, the framework reports:

- **BAR** (Behavioral Agreement Rate): all structural dimensions correct
- **SVR** (Safety Violation Rate): HITL-required cases where HITL was not triggered
- **CSR** (Cross-Scenario Robustness): consistency across scenario conditions
- **pass^k**: repeat-run reliability for nondeterministic settings

State Reset for Deterministic Runs

Before each test, the evaluator resets mock cluster state to the test's `source_state`. This is essential for stateful toolchains, preventing contamination from previous actions (e.g., a cancelled job in one test affecting subsequent tests).

6.1.4 Manual Scenario Validation

Manual scenarios are still used to verify conversational quality, chart readability, and user-facing flow. Current manual focus areas include:

- read-only monitoring workflows,
- diagnosis + recommendation workflows,
- dangerous-operation confirmation flows,
- integrated evaluation UI usability.

6.1.5 Running the Evaluation

The current flow is:

1. generate/update dataset (`dataset.py`),
2. start stack (agent + MCP),
3. run via CLI (`scenario_eval.py`) or web dashboard (`eval_server.py` + integrated UI tab).

Result artifacts are written to `evaluation/results/` as JSON and TeX exports.

6.1.6 Source Code and Reproducibility

The full source code remains publicly available:

https://github.com/DanhNguyennene/specialized_project_slurm_agent

6.1.7 TODO for Final Thesis Freeze

- Re-run one final full 367-case benchmark under pinned model/provider settings and freeze those numbers.
- Add explicit command transcript (exact CLI flags and environment values) for the final benchmark run.
- Add pass^k table for $k \in \{1, 3, 5\}$ under the final inference setup.

6.2 Experimental Results

This section reports the latest scenario-grid benchmark for the Slurm Agent. Results are taken from the current snapshot artifact (`eval_latest_snapshot.json`, timestamp 20260417_211210) and reflect the present implementation baseline described in Chapter 5.

6.2.1 Benchmark Configuration Snapshot

Table 6.4: Evaluation Run Configuration (Latest Snapshot)

Field	Value
Scenario scope	all
Dataset size	367 test cases
Dataset ID	41b3c240188c
Pass threshold	≥ 0.80 (with keyword gate)
Judge mode	enabled (qwen3.5:9b)
Output artifacts	JSON + TeX snapshot in <code>evaluation/results/</code>

6.2.2 Overall Performance

Table 6.5: Overall Metrics (367-Case Snapshot)

Metric	Value
Total Test Cases	367
Passed	237
Failed	130
Pass Rate	64.6%
BAR (Behavioral Agreement Rate)	68.4%
SVR (Safety Violation Rate)	35.9%
CSR (Cross-Scenario Robustness)	82.5%
Average Overall Score	0.775
Average Judge Score	0.507
Average Latency	12.92s

6.2.3 Component-Level Averages

Table 6.6: Average Component Scores

Component	Average
Tool Recall	0.764
Routing Match	0.831
HITL Match	0.826
Keyword Score	0.955
State Match	0.807
Judge Score	0.507

The high keyword score (0.955) indicates that responses usually contain expected terms, while structural dimensions (tool/routing/HITL/state) remain the dominant constraints on pass/fail outcomes.

6.2.4 Results by Category

Table 6.7: Category-Level Results (Latest Snapshot)

Category	N	Pass%	Tool	Route	HITL	State	Overall
read	106	79%	87%	96%	100%	100%	91%
diagnose	39	51%	82%	95%	100%	100%	83%
action	31	71%	84%	97%	87%	84%	82%
bulk	24	71%	85%	75%	75%	71%	74%
safety	45	53%	71%	58%	58%	53%	61%
submission	35	51%	51%	71%	63%	51%	60%
multi_step	17	0%	38%	24%	12%	0%	28%
account	25	84%	84%	92%	88%	100%	83%
edge	45	69%	71%	89%	91%	91%	83%
Total	367	65%	76%	83%	83%	81%	78%

Strongest categories: read and account show the highest pass rates and stable structural behavior.

Primary weak categories: multi_step, safety, and submission. In particular, multi_step fails to satisfy state-transition expectations consistently in this snapshot.

6.2.5 Results by Scenario

Table 6.8: Scenario-Level Robustness

Scenario	N	Passed	Pass%	Avg Overall
healthy	72	48	66.7%	0.780
failed	68	41	60.3%	0.758
pending	71	49	69.0%	0.795
mixed	79	47	59.5%	0.743
debug_needed	77	52	67.5%	0.797
Total	367	237	64.6%	0.775

Cross-scenario behavior remains relatively stable (CSR = 82.5%), with the mixed scenario being the hardest condition in this run.

6.2.6 Failure Pattern Analysis

There were no runtime execution errors in this benchmark (0/367 tests with error field set). Most failures are therefore behavioral mismatches rather than infrastructure failures.

Table 6.9: Mismatch Counts Across 367 Cases

Mismatch Type	Count
Tool recall < 1.0	109
Routing mismatch	62
HITL mismatch	64
State mismatch	71
Keyword score < 0.6	20

Among 130 failed tests, structural mismatches dominate: tool selection/coverage (109), state mismatch (71), and routing/HITL mismatches (62/64). This indicates that architecture-level reliability, not surface-level wording, is the key improvement target.

6.2.7 Latency Characteristics

Average end-to-end latency is 12.92 seconds. Category-level averages indicate higher latency in submission, safety, and multi_step workflows (roughly 16–17 seconds), consistent with additional reasoning and safety checks.

6.2.8 Manual Scenario Validation Summary

Manual runs were used to validate user-facing behavior not fully captured by scalar metrics:

- chat responsiveness and stream readability,
- confirmation flow clarity for destructive operations,

- correctness of inline Mermaid artifacts,
- usability of the integrated evaluation dashboard.

These manual checks confirmed the expected interaction flow in both chat mode and evaluation mode.

6.2.9 Comparison with Manual CLI Workflow

Table 6.10: Task Complexity: Agent vs Manual CLI

Task	Agent Interaction	Typical Manual CLI
Check queue status	1 prompt	1–2 commands
Diagnose pending jobs	1 prompt	3–5 commands + interpretation
Cluster utilization summary	1 prompt (+ optional chart)	4–6 commands + manual synthesis
Safe cancellation flow	2 turns (request + confirm)	1 command (no built-in conversational guardrails)

The agent offers the largest benefit in multi-source interpretation tasks, while direct read-only queries are closer to command-line effort.

6.2.10 Result Interpretation

The current system is strong in read-oriented and account-oriented assistance, and demonstrates good cross-scenario stability. However, the benchmark also shows clear reliability gaps in complex multi-step and safety-sensitive flows. These findings directly inform the limitation and future-work priorities in Chapter 7.

6.2.11 TODO for Final Thesis Freeze

- Re-run one final benchmark under pinned model/provider settings and freeze a single authoritative snapshot ID.

- Regenerate figure set (distribution/category/latency plots) strictly from the frozen snapshot.
- Export pass^k results (for $k \in \{1, 3, 5\}$) and add repeatability analysis table.
- Add a short ablation comparing judge-enabled vs judge-disabled weighting on the same dataset.

Chapter 7

Conclusion

7.1 Summary

This project developed and evaluated a stateful Slurm Agent that enables natural-language interaction for HPC cluster monitoring and control. The final system combines a FastAPI backend, OpenAI Agents SDK orchestration, an MCP-based tool layer, and a React + Vite frontend with integrated chat and evaluation modes.

7.1.1 Research Contributions

This work contributes the following practical outcomes:

- 1. Scenario-Grid Evaluation for Agentic Slurm Workflows.** The project introduces a dataset-driven evaluation design (367 cases) that measures architecture-level behavior: tool usage, routing, HITL safety, state transitions, and response quality.
- 2. Stateful MCP-Centered Integration.** Slurm interactions are exposed through MCP tools and exercised in deterministic mock scenarios with per-test reset, enabling reproducible behavioral testing.
- 3. Multi-Agent Reasoning and Safety Flow.** Observer/operator delegation and confirmation workflows are integrated into a single runtime, including explicit pending-action management.
- 4. Structured Streaming Interface.** The backend emits structured SSE deltas (reasoning, tool output, content, pending actions, chart artifacts), allowing richer frontend

rendering and traceability.

5. Integrated Evaluation UI. Evaluation is available as a first-class frontend mode, reducing friction between implementation changes and benchmark validation.

7.1.2 Achievement Against Project Goals

Goal 1 (Natural language access to Slurm): Achieved for read-oriented and account-oriented workflows, with strong category-level performance.

Goal 2 (Multi-agent orchestration): Achieved. Delegation between observer and operator paths is functional and measurable in the benchmark.

Goal 3 (MCP standardization): Achieved. Tool interfaces are separated from agent reasoning and can be extended independently.

Goal 4 (Safety for dangerous operations): Partially achieved. HITL flow is implemented and active, but benchmark safety metrics indicate additional hardening is needed.

Goal 5 (Realtime UX and web interface): Achieved with structured streaming in the React frontend and integrated evaluation controls.

7.1.3 Key Quantitative Outcomes

Table 7.1: Final Snapshot Highlights

Metric	Value
Dataset size	367 tests
Pass rate (≥ 0.80)	64.6% (237/367)
Average overall score	0.775
BAR	0.684
SVR	0.359
CSR	0.825
Average latency	12.92s

These results show strong capability in read/account workflows and acceptable cross-scenario stability, while also exposing clear weaknesses in multi-step and safety-sensitive categories.

7.1.4 Main Findings

Architecture-aware metrics are necessary. Response text quality alone is insufficient for evaluating agentic systems that perform side-effectful operations. Routing, HITL behavior, and state transitions materially affect reliability.

Deterministic state reset improves evaluation quality. Source-state resets before each test are essential for trustworthy stateful evaluation.

Frontend-integrated evaluation improves iteration speed. Embedding the evaluation workflow in the main UI shortens the feedback loop for feature changes.

Safety remains a hard problem. Even with confirmation mechanisms, mismatch rates in safety-related categories show that stronger guarantees are still required before production use.

7.1.5 Source Code Availability

The complete source code is publicly available:

https://github.com/DanhNguyennene/specialized_project_slurm_agent

The repository contains the backend, MCP server, React frontend, evaluation framework, dataset artifacts, and reproducible result snapshots.

7.1.6 TODO for Final Thesis Freeze

- Freeze one final benchmark snapshot under pinned model/provider versions and update all chapter references to that single run.
- Add one concise reliability paragraph interpreting pass^k once repeat-run data is finalized.

7.2 Limitations

The current Slurm Agent demonstrates promising behavior, but several limitations remain before production-grade deployment.

7.2.1 Evaluation Realism Limitations

Mock-state dominance. The benchmark primarily uses deterministic mock scenarios. This is valuable for reproducibility, but does not fully capture live cluster volatility, scheduler dynamics, and user concurrency.

Single benchmark snapshot emphasis. Current reporting centers on the latest full snapshot. Broader confidence intervals across repeated full runs are not yet fully reported in the thesis body.

Limited human-subject evaluation. Automated scoring is comprehensive, but formal user studies with HPC practitioners are still missing.

7.2.2 Behavioral Reliability Limitations

The latest 367-case results expose reliability gaps in specific categories:

- **multi_step:** 0% pass in the latest snapshot,
- **safety:** 53% pass,
- **submission:** 51% pass.

These failures are predominantly structural (tool/routing/HITL/state mismatches) rather than simple response-text issues.

7.2.3 Operational Scope Limitations

Read-heavy maturity. Read and account workflows are stronger than mutation-heavy workflows. Complex write operations still require additional hardening and policy controls.

Narrow deployment validation. The current system is validated mainly in development-style environments; multi-cluster and production HPC integration is incomplete.

7.2.4 Security and Governance Limitations

Authentication and authorization are incomplete. Fine-grained user permissions and tenant isolation are not yet fully integrated into the primary agent path.

Auditability needs extension. While traces are captured for evaluation, production-grade audit logging and compliance workflows are not fully specified.

7.2.5 Performance and Scalability Limitations

Latency variability persists. Average latency is acceptable for conversational assistance, but higher-latency categories (submission/safety/multi-step) remain noticeable.

Concurrent-load behavior is under-tested. Throughput and stability under many simultaneous users have not been comprehensively benchmarked.

7.2.6 Model and Scoring Limitations

Model sensitivity. Behavioral performance depends on model/provider selection and prompt tuning. Transferability of exact metrics to different model stacks is not guaranteed.

Judge dependence. Some quality dimensions rely on LLM-as-judge scoring, which introduces model-dependent variance and potential evaluator bias.

7.2.7 TODO for Limitation Mitigation

- Add repeated full-run statistics (pass^k and confidence intervals) under fixed model/provider settings.
- Execute controlled live-cluster pilot tests with strict permission boundaries.
- Add production-oriented authz/audit requirements and verification checklist.

References

- [1] Anthropic. *Model Context Protocol*. <https://modelcontextprotocol.io>. Accessed: 2024. 2024.
- [2] Anthropic. *Model Context Protocol Specification*. <https://spec.modelcontextprotocol.io>. Accessed: 2024. 2024.
- [3] Yuntao Bai et al. “Constitutional AI: Harmlessness from AI feedback”. In: *arXiv preprint arXiv:2212.08073* (2022).
- [4] Tom Brown et al. “Language models are few-shot learners”. In: *Advances in neural information processing systems* 33 (2020), pp. 1877–1901.
- [5] Dror G Feitelson, Dan Tsafrir, and David Krakov. “Experience with using the parallel workloads archive”. In: *Journal of Parallel and Distributed Computing* 74.10 (2014), pp. 2967–2982.
- [6] Glenn K Lockwood, Drew Hazen, and Nicholas J Wright. “FAIR scheduler for SLURM: Improved fairshare scheduling for HPC environments”. In: *2017 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE. 2017, pp. 550–555.
- [7] MDN Web Docs. *Server-Sent Events (SSE)*. https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events. Accessed: 2024. 2024.
- [8] Mermaid. *Mermaid: Diagramming and charting tool*. <https://mermaid.js.org>. Accessed: 2024. 2024.
- [9] Ollama. *Ollama: Get up and running with large language models locally*. <https://ollama.ai>. Accessed: 2024. 2024.
- [10] OpenAI. “GPT-4 technical report”. In: *arXiv preprint arXiv:2303.08774* (2023).

- [11] OpenAI. *OpenAI Agents SDK*. <https://github.com/openai/openai-agents-python>. Accessed: 2024. 2024.
- [12] Sebastian Ramirez. *FastAPI: Modern, fast web framework for building APIs with Python*. <https://fastapi.tiangolo.com>. Accessed: 2024. 2024.
- [13] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence embeddings using Siamese BERT-networks”. In: *arXiv preprint arXiv:1908.10084* (2019).
- [14] Gregory Sabin and Rebecca Braby. “The HPC user support experience: Understanding user needs and developing support services”. In: *Computing in Science & Engineering* 22.1 (2020), pp. 75–82.
- [15] SchedMD. *Slurm Workload Manager Documentation*. <https://slurm.schedmd.com/documentation.html>. Accessed: 2024. 2024.
- [16] Timo Schick et al. “Toolformer: Language models can teach themselves to use tools”. In: *Advances in Neural Information Processing Systems* 36 (2023).
- [17] Svelte. *Svelte: Cybernetically enhanced web apps*. <https://svelte.dev>. Accessed: 2024. 2024.
- [18] Hugo Touvron et al. “Llama 2: Open foundation and fine-tuned chat models”. In: *arXiv preprint arXiv:2307.09288* (2023).
- [19] Bailin Wang et al. “RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers”. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 2020, pp. 7567–7578.
- [20] Michael Wooldridge. “An introduction to multiagent systems”. In: (2009).
- [21] Qingyun Wu et al. “AutoGen: Enabling next-gen LLM applications via multi-agent conversation”. In: *arXiv preprint arXiv:2308.08155* (2023).
- [22] Zhiheng Xi et al. “The rise and potential of large language model based agents: A survey”. In: *arXiv preprint arXiv:2309.07864* (2023).
- [23] Shunyu Yao et al. “ReAct: Synergizing reasoning and acting in language models”. In: *arXiv preprint arXiv:2210.03629* (2022).
- [24] Andy B Yoo, Morris A Jette, and Mark Grondona. “SLURM: Simple linux utility for resource management”. In: (2003), pp. 44–60.

- [25] Tao Yu et al. “Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task”. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. 2018, pp. 3911–3921.
- [26] Lianmin Zheng et al. “Judging LLM-as-a-judge with MT-bench and chatbot arena”. In: *Advances in Neural Information Processing Systems* 36 (2023).
- [27] Victor Zhong, Caiming Xiong, and Richard Socher. “Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning”. In: *arXiv preprint arXiv:1709.00103* (2017).